

Books & Authors Management System

Spring Boot + JPA + JSP — Project Write-up

1. Overview

This project is a Spring Boot MVC web application that manages two related entities — **Author** and **Book** — and supports Create, Read and Update operations through JSP views styled with custom CSS. Data is persisted in an H2 in-memory database via Spring Data JPA. On startup the database is automatically populated with 10 authors and 10 books to demonstrate every feature.

2. Entity Relationship Design

The two entities share a one-to-many / many-to-one relationship: a single Author can write many Books, and each Book belongs to exactly one Author.

AUTHOR		1	N	BOOK	
id (PK, auto)		<-----		id (PK, auto)	
name (unique, NOT NULL)				title (NOT NULL)	
country				genre	
				published_year	
				author_id (FK -> AUTHOR.id, NOT NULL)	

JPA mapping highlights:

- **Author:** @OneToMany(mappedBy="author", cascade=ALL, orphanRemoval=true)
- **Book:** @ManyToOne(optional=false) @JoinColumn(name="author_id")
- Validation via @NotBlank and @NotNull.

3. Layered Architecture

The application follows the standard Spring layered architecture:

- **Entity layer** — JPA-annotated POJOs (Author, Book).
- **Repository layer** — interfaces extending JpaRepository; BookRepository declares a custom JPQL inner-join query.
- **Service layer** — AuthorService, BookService encapsulate business logic and delegate persistence.
- **Controller layer** — Spring MVC controllers map HTTP requests to JSP views.
- **View layer** — JSP pages with JSTL/EL plus a shared CSS file.

4. Implementation Details

4.1 Populate Database (DataLoader)

A CommandLineRunner bean inserts 10 authors and 10 books on startup.

```
@Component
public class DataLoader implements CommandLineRunner {
    public void run(String... args) {
        if (authorRepository.count() > 0) return;
        List<Author> authors = List.of(
            new Author("J.K. Rowling", "United Kingdom"),
            new Author("George R.R. Martin", "United States"),
```

```

        ... // 10 total
    );
    authorRepository.saveAll(authors);
    bookRepository.saveAll(List.of( ... 10 books ... ));
}
}

```

4.2 Create Operation

JSP form (book_form.jsp) posts to the controller, which validates the bean and saves it via the service.

```

@PostMapping
public String create(@Valid @ModelAttribute("book") Book book,
                    BindingResult br, Model model) {
    if (br.hasErrors()) {
        model.addAttribute("authors", authorService.findAll());
        return "book_form";
    }
    bookService.save(book);
    return "redirect:/books";
}

```

Integrity violations (e.g. duplicate author names due to the unique constraint) are caught by a global `@ControllerAdvice` that renders a friendly error page:

```

@ExceptionHandler(DataIntegrityViolationException.class)
public String handleIntegrity(DataIntegrityViolationException ex, Model model) {
    model.addAttribute("errorMessage",
        "Data integrity violation: " + ex.getMostSpecificCause().getMessage());
    return "error";
}

```

4.3 Read Operation + Custom INNER JOIN

The repository defines a JPQL query that performs an inner join between Book and Author:

```

@Query("SELECT b.title, b.genre, b.publishedYear, a.name, a.country " +
        "FROM Book b INNER JOIN b.author a " +
        "ORDER BY a.name, b.title")
List<Object[]> findBooksWithAuthors();

```

The Books JSP iterates over the result and renders both the standard list and the joined list:

```

<c:forEach var="row" items="${joined}">
    <tr>
        <td>${row[0]}${row[1]}${row[2]}${row[3]}${row[4]}

```

4.4 Update Operation

```

@GetMapping("/edit/{id}")
public String edit(@PathVariable Long id, Model model) {
    model.addAttribute("book", bookService.findById(id));
    model.addAttribute("authors", authorService.findAll());
    return "book_form";
}

@PostMapping("/update/{id}")
public String update(@PathVariable Long id,
                    @Valid @ModelAttribute("book") Book book,
                    BindingResult br, Model model) {
    if (br.hasErrors()) { ... }
    book.setId(id);
    bookService.save(book);
    return "redirect:/books";
}

```

5. Testing

Two test classes verify the repository and service layers:

- **BookRepositoryTest** — uses `@DataJpaTest` to spin up a real H2 instance and asserts that the custom inner-join query returns the expected columns.
- **BookServiceTest** — uses Mockito to mock the repository and verifies that `findAll()` and `save()` delegate correctly.

Run with `mvn test`.

6. Screenshots

After running `mvn spring-boot:run` open the URLs below and capture screenshots to embed in your final submission:

- **Home**: `http://localhost:8080/`
- **Authors list**: `http://localhost:8080/authors`
- **Add Author form**: `http://localhost:8080/authors/new`
- **Books list + INNER JOIN table**: `http://localhost:8080/books`
- **Add / Edit Book form**: `http://localhost:8080/books/new`
- **H2 console**: `http://localhost:8080/h2-console`

7. Challenges & Solutions

Challenge 1 — Serving JSP from Spring Boot 3. Spring Boot 3 requires Jakarta EE 9+ APIs. Solved by adding `tomcat-embed-jasper` plus the Jakarta JSTL API and Glassfish JSTL implementation, and configuring `spring.mvc.view.prefix/suffix`.

Challenge 2 — Binding nested objects in forms. The Book form needs to set an Author by id. Solved by binding to `author.id` in the JSP `<form:select>`; Spring's data binder resolves it automatically.

Challenge 3 — Integrity-violation handling. The unique constraint on author name would otherwise produce a 500 stack trace. Solved with a global `@ControllerAdvice` rendering a friendly error JSP.

Challenge 4 — JPQL inner join syntax. Used the relationship attribute (`b.author`) instead of an explicit ON clause so JPA generates the correct SQL inner join.

8. GitHub Repository

Push the project to GitHub and replace this line with your repo URL:

<https://github.com/<your-username>/bookapp>